INSTITUT SUPÉRIEUR DE GESTION DE TUNIS
Since 1969

RAPPORT DU PROJET PWEB

RAPPORT PRÉSENTÉ PAR :
GHOFRANE ARAAR & EYA INB ABDELHEDI
& AMIRA MIDANI

2LNBC-BIS1

ANNÉE UNIVERSITAIRE 2025/2026
DEUXIEME SEMESTRE

Sommaire

1. Présentation générale du projet
2. Objectifs
3. Architecture et technologies utilisées
4. Structure du projet
5. Base de données
6. Fonctionnalités
7. Les interfaces
8. Validation des données
9. Analyse critique et points d'amélioration
10. Références
11. Conclusion

Présentation générale du projet

UberCueillette est une application web developpee dans le cadre du mini-projet Web2 a l'Institut Superieur de Gestion (ISG) de Tunis pour l'annee academique 2025-2026. S'inspirant du modele de mise en relation Uber, la plateforme connecte les agriculteurs qui publient des besoins de main d'oeuvre saisonniere et les ouvriers agricoles qui cherchent des missions remunerees pour la cueillette d'olives, d'agrumes, de tomates et d'autres fruits.

Le projet repond a une problematique reelle : les agriculteurs tunisiens peinent a recruter de la main d'oeuvre lors des pics de recolte, tandis que des milliers de travailleurs journaliers cherchent des missions sans canal organise. UberCueillette centralise et automatise cette mise en relation sur un portail web unique.

Objectifs

Objectifs fonctionnels:

- Permettre aux agriculteurs de publier des offres de récolte en quelques clics
- Permettre aux ouvriers de consulter, filtrer et postuler aux offres disponibles
- Gérer le cycle complet : candidature -> acceptation/refus -> évaluation post-récolte
- Assurer la traçabilité des gains et des notes de chaque ouvrier par mission
- Couvrir les 24 gouvernorats tunisiens avec un système de fruits extensible dynamiquement

Objectifs pédagogiques:

- Développer une application web dynamique complète en PHP/MySQL
- Maîtriser la gestion des sessions et l'authentification par rôle utilisateur
- Sécuriser les données avec PDO (requêtes préparées) et bcrypt (hachage des mots de passe)
- Concevoir et implémenter un schéma de base de données relationnel (clés étrangères, contraintes)
- Créer une interface utilisateur en HTML5/CSS3 pur
- Appliquer une architecture modulaire organisée par rôle (agriculteur/ouvrier)

Architecture et technologies utilisées

1.Stack technique:

Couche	Technologie
Backend	PHP 8.2
Frontend	HTML5 + CSS3+JavaScript
Base de données	MySQL
Acces BDD	PDO (PHP Data Objects)
Serveur web	Apache (XAMPP)

2.Flux utilisateur complet:

Etape	Action	Destination
Acces public	Visite de la page d'accueil	index.php
Inscription	Formulaire agriculteur ou ouvrier	inscri.php
Connexion	Saisie pseudo + MDP + role	connexion.php
Agriculteur	Publication offre, gestion candidatures	menu_agriculteur.php + sous-pages
Ouvrier	Parcourir offres, postuler, suivre	menu.php + lister.php + candidatures
Post-recolte	Evaluation, remuneration, gains	noter.php + ouvriernotecomm.php
Deconnexion	session_unset() + session_destroy()	Retour a index.php

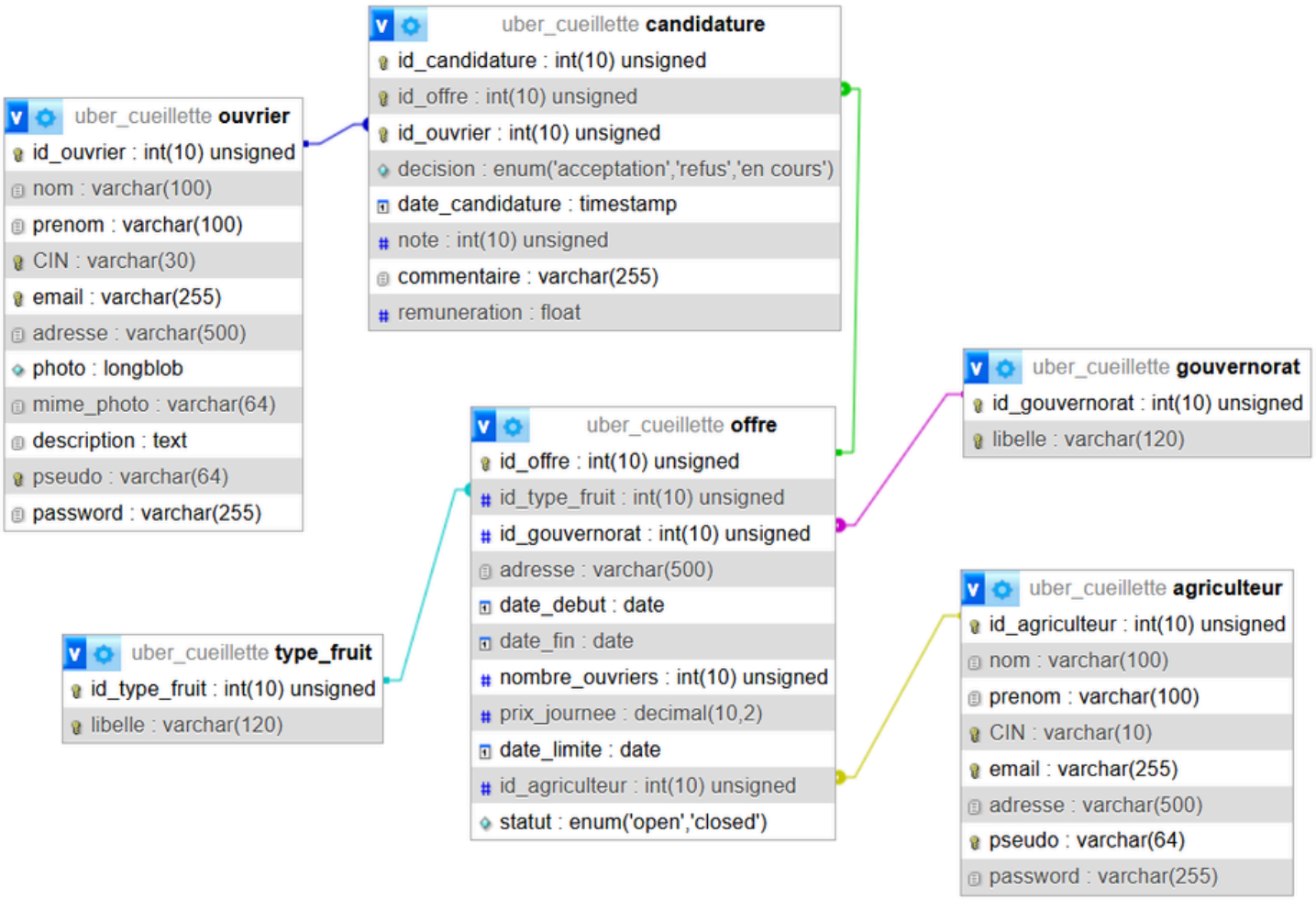
Structure du projet

Le projet UberCueillette adopte une architecture modulaire organisée par rôle utilisateur. La racine contient les fichiers d'entrée communs (**index.php, db.php**). Le dossier **agriculteur/** regroupe toutes les fonctionnalités de l'agriculteur, de l'inscription jusqu'à l'évaluation des ouvriers. Le dossier **ouvrier/** contient les pages dédiées à l'ouvrier, de l'inscription au suivi des candidatures. Le dossier **offre/** gère la liste publique des offres, tandis que **css/** et **java/** centralisent respectivement le style et la validation JavaScript. Cette organisation garantit une séparation claire des responsabilités et une maintenance facilitée.

```
uber_cueillette/
  index.php                <- Page d'accueil principale
  config/db.php            <- Connexion PDO a la base de donnees
  agriculteur/
    inscri.php             <- Inscription agriculteur
    connexion.php          <- Connexion agriculteur
    menu_agriculteur.php   <- Menu apres connexion
    proposeroffre.php      <- Ajouter une offre de recolte
    voirmesoffre.php       <- Liste + suppression des offres
    voircandidatures.php   <- Consulter et gerer les candidatures
    decision.php           <- Accepter ou refuser (POST)
    noter.php              <- Evaluer un ouvrier
    voirprofil.php         <- Voir le profil
    modifierprofil.php     <- Modifier le profil
  ouvrier/
    inscri.php             <- Inscription ouvrier (avec photo)
    connexion1.php         <- Connexion ouvrier
    menu.php               <- Menu apres connexion
    ouvrierprofil.php      <- Voir le profil + moyenne
    modifierprofil.php     <- Modifier le profil
    mescandidatures.php    <- Suivi des candidatures
    ouvriernotecomm.php    <- Chantiers termines + gains
  offre/Lister.p           <- Postuler a une offre
  css/style.css            <- Liste publique des offres + filtres
  java/script.js           <- Validation JS et filtres
```

Base de données

La base de données uber_cueillette est gerée sous MySQL. Elle contient 6 tables reliées par des clés étrangeres avec comportements CASCADE, des index de performance sur les colonnes de jointure fréquentes, et des contraintes UNIQUE pour garantir l'intégrite des données.



Base de données

Connexion a la Base de Données (PDO):

Conformement au cours, la connexion a MySQL se fait via l'extension PDO (PHP Data Objects) avec gestion d'exception. Le fichier config/db.php est inclus dans toutes les pages via include("../config/db.php").

Fichier config/db.php:

uber_final3 > config > db.php

```php
<?php
$host = "localhost";
$dbname = "uber_cueillette";
$user = "root";
$pass = "";

try {
    $conn = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8", $user, $pass);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $conn->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    die("Erreur connexion : " . $e->getMessage());
}
?>
```

Inclusion dans chaque page:

exemple dans le fichier agriculteur/decision.php

uber_final3 > agriculteur > decision.php

```php
<?php
session_start();
include("../config/db.php");
```

exemple dans le fichier ouvrier/incsri.php

uber_final3 > ouvrier > inscri.php

```php
<?php
session_start(); include("../config/db.php");
```

Fonctionnalités

Gestion des Sessions PHP:

Fichier config/db.php:

uber_final3 > connexion.php

```php
<?php
session_start();
// Rediriger si deja connecte
if (isset($_SESSION['id_agriculteur'])) { header("Location: agriculteur/menu_agriculteur.php"); exit; }
if (isset($_SESSION['id_ouvrier']))     { header("Location: ouvrier/menu.php"); exit; }

include("config/db.php");
$message = "";

if (isset($_POST['login'])) {
    $pseudo   = trim($_POST['pseudo']);
    $password = $_POST['password'];
    $role     = $_POST['role'];

    if ($role === 'agriculteur') {
        $stmt = $conn->prepare("SELECT * FROM agriculteur WHERE pseudo = ?");
        $stmt->execute([$pseudo]);
        $user = $stmt->fetch();
        if ($user && password_verify($password, $user['password'])) {
            $_SESSION['id_agriculteur'] = $user['id_agriculteur'];
            header("Location: agriculteur/menu_agriculteur.php"); exit;
        } else { $message = "Pseudo ou mot de passe incorrect."; }

    } elseif ($role === 'ouvrier') {
        $stmt = $conn->prepare("SELECT * FROM ouvrier WHERE pseudo = ?");
        $stmt->execute([$pseudo]);
        $user = $stmt->fetch();
        if ($user && password_verify($password, $user['password'])) {
            $_SESSION['id_ouvrier'] = $user['id_ouvrier'];
            header("Location: ouvrier/menu.php"); exit;
        } else { $message = "Pseudo ou mot de passe incorrect."; }
    }
}
?>
```

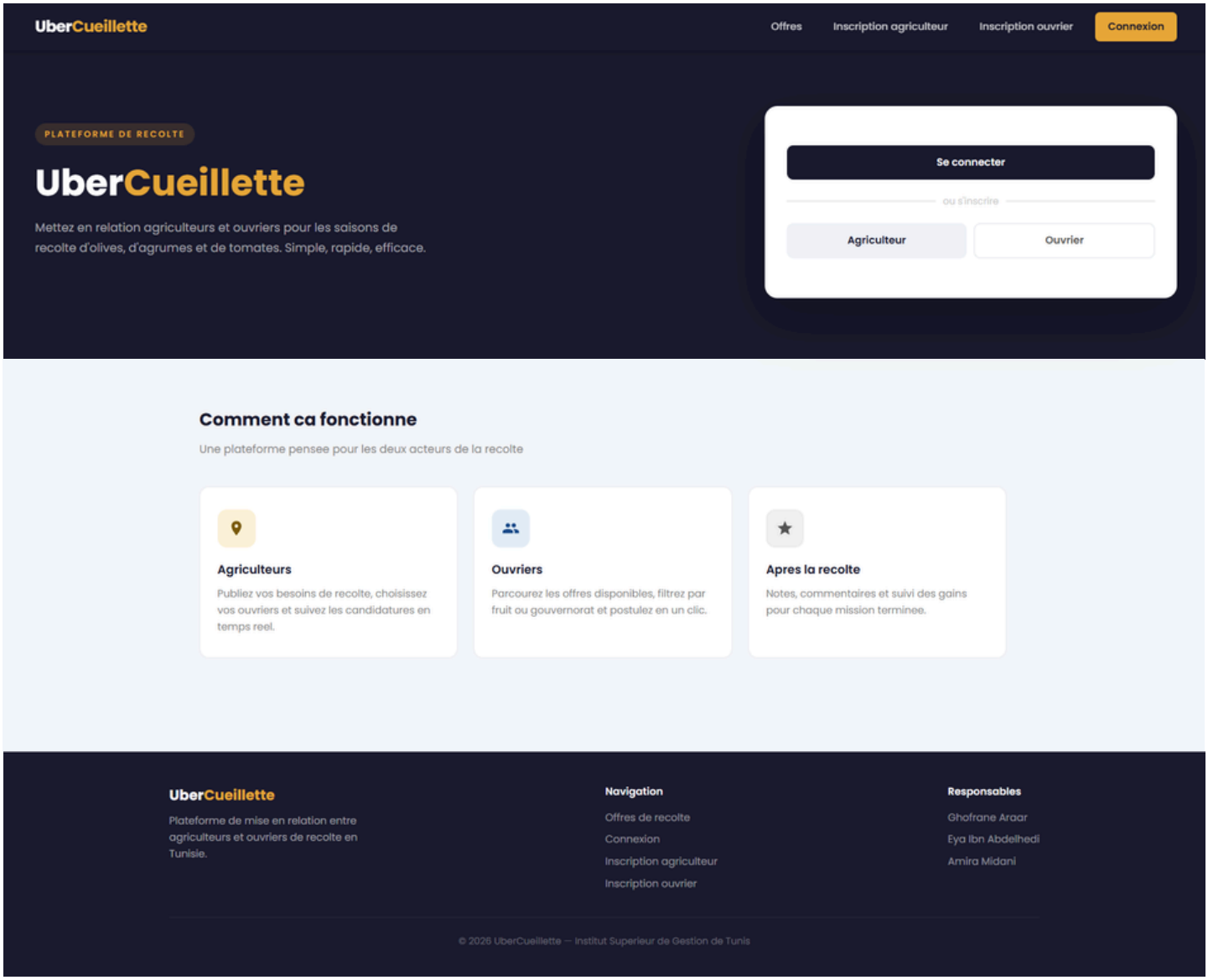
Deconnexion propre:

```php
if (isset($_GET['logout'])) {
    session_unset();
    session_destroy();
}
```

Les interfaces

Page Accueil (index.php):

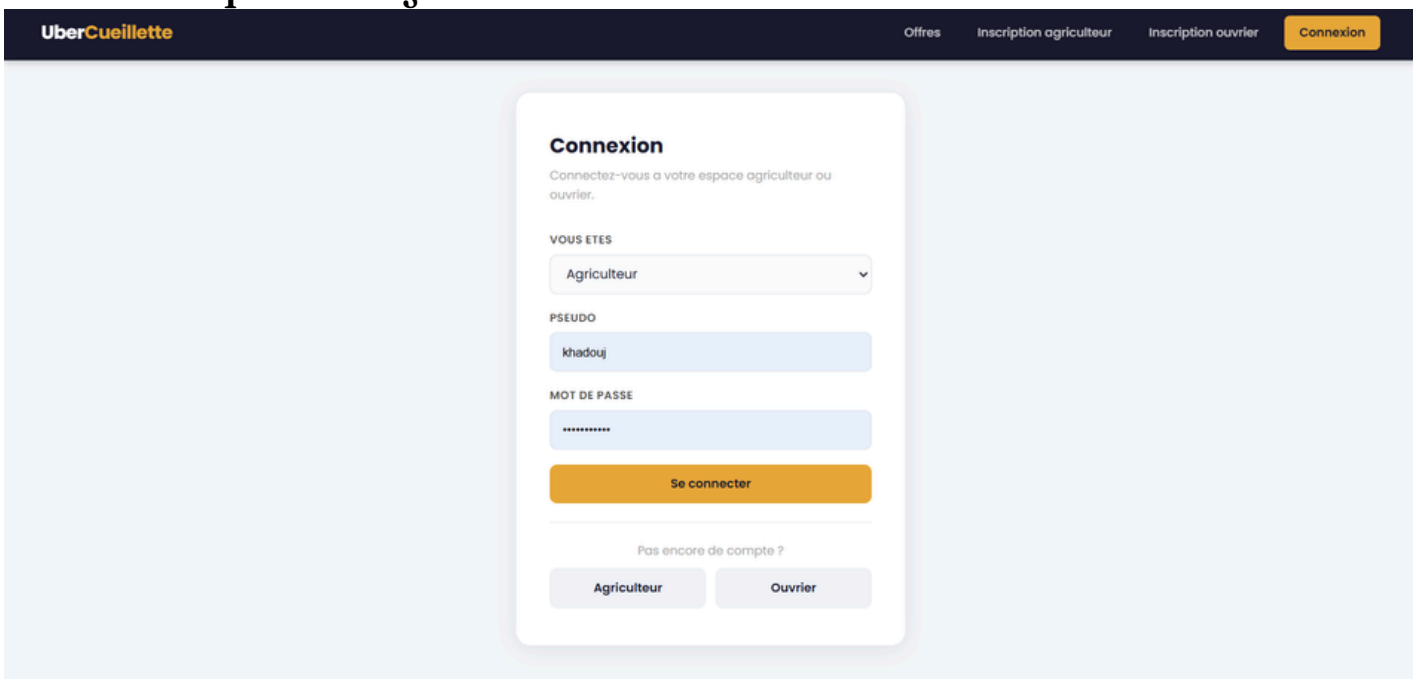
Section Hero avec titre, description et carte d'accès membre.
Section Features avec 3 cartes explicatives (Agriculteurs, Ouvriers, Apres récolte).
Section Footer qui contient les liens des comptes LinkedIn des responsables et les liens des autres pages.



Les interfaces

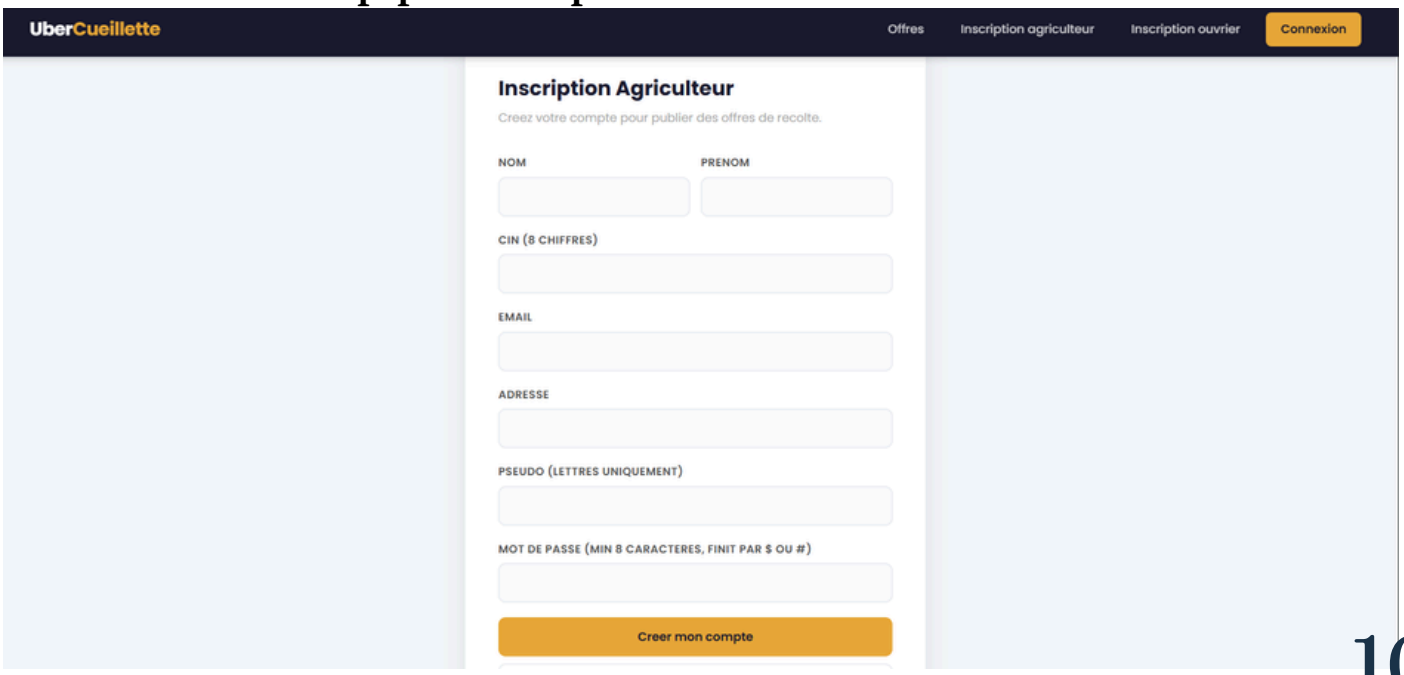
Page de connexion (connexion.php):

Formulaire unifie avec selecteur de role, champs pseudo et mot de passe. Liens d'inscription visibles en bas. Redirection automatique si deja connecte.



Inscription Agriculteur (agriculteur/inscri.php):

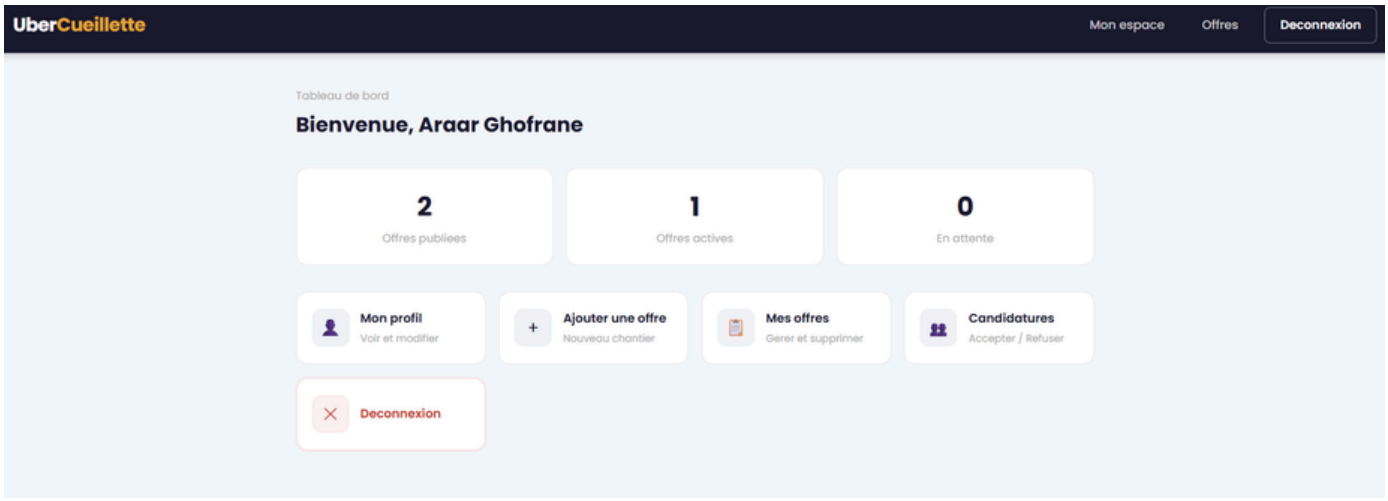
Formulaire avec validation temps réel (CIN 8 chiffres, pseudo lettres, MDP). Hachage bcrypt à la soumission. Identique pour l'ouvrier + champ photo upload.



Les interfaces

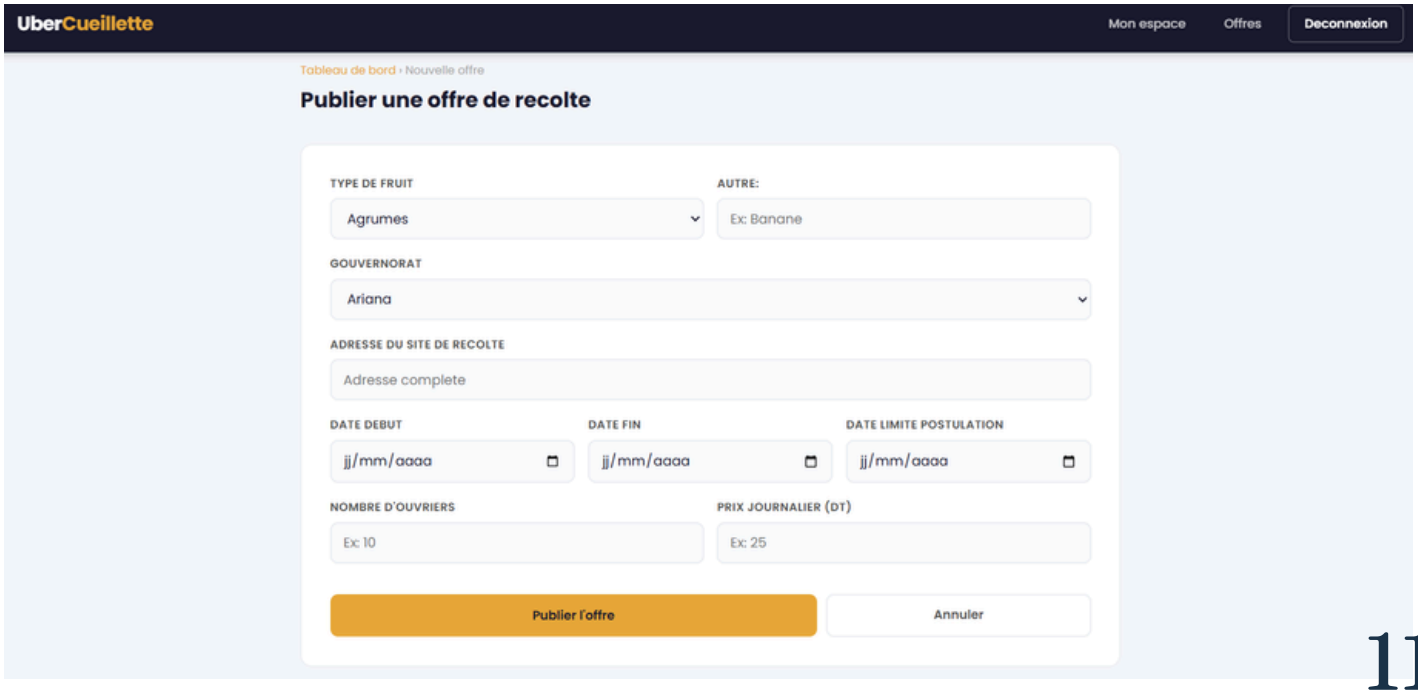
Tableau de bord Agriculteur (menu_agriculteur.php):

3 statistiques calculees en temps reel par requetes SQL + 6 tuiles de navigation vers les fonctionnalites (profil, offres, candidatures, deconnexion).



Publication d'offre (proposeroffre.php):

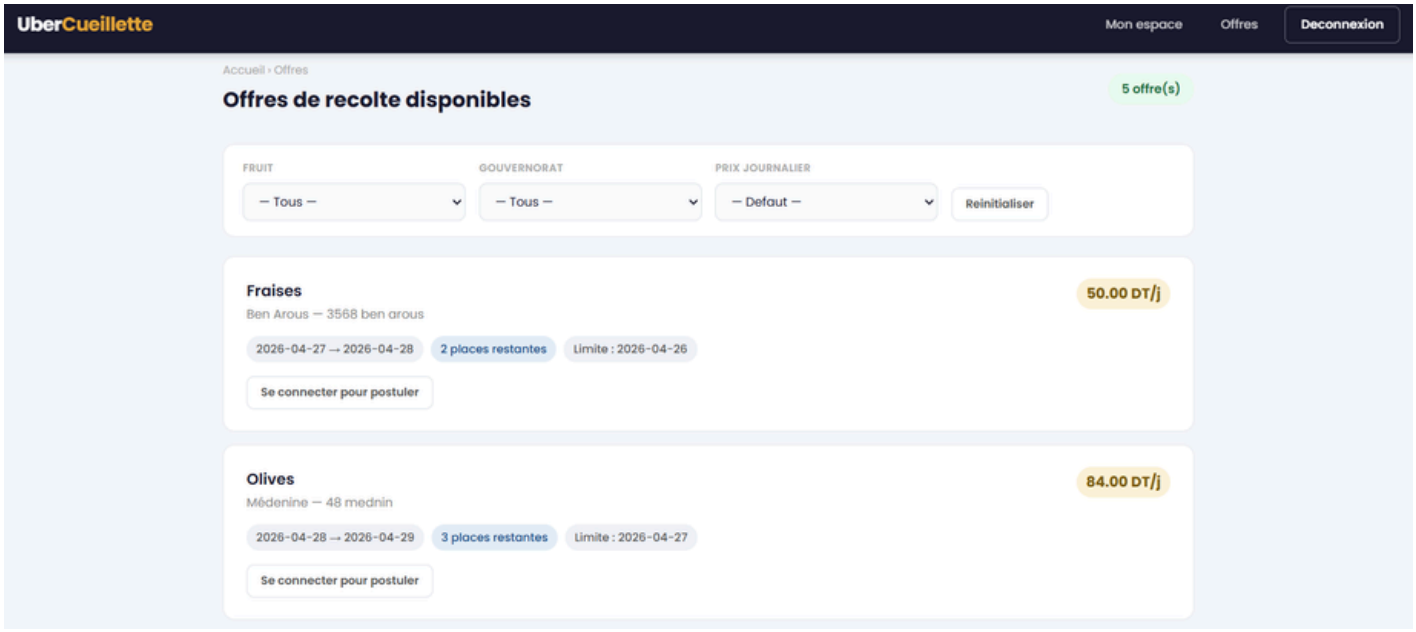
Fruit et gouvernorat selectionnable ou creables dynamiquement. 3 dates avec validation PHP + JS. Nombre d'ouvriers et prix en DT.



Les interfaces

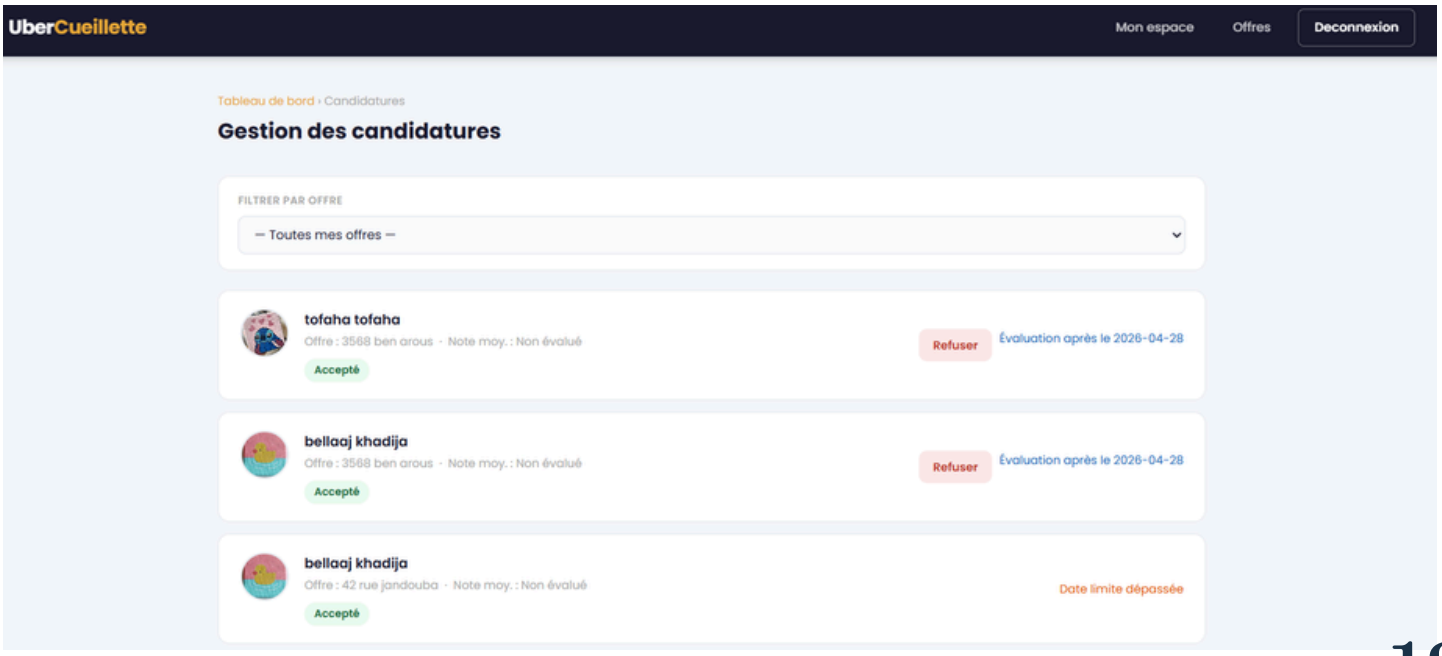
Offres disponibles (offre/lister.php):

Page publique avec filtres auto-soumis (fruit, gouvernorat, prix). Seules les offres actives et non-completes sont affichees.



Gestion candidatures (voircandidatures.php):

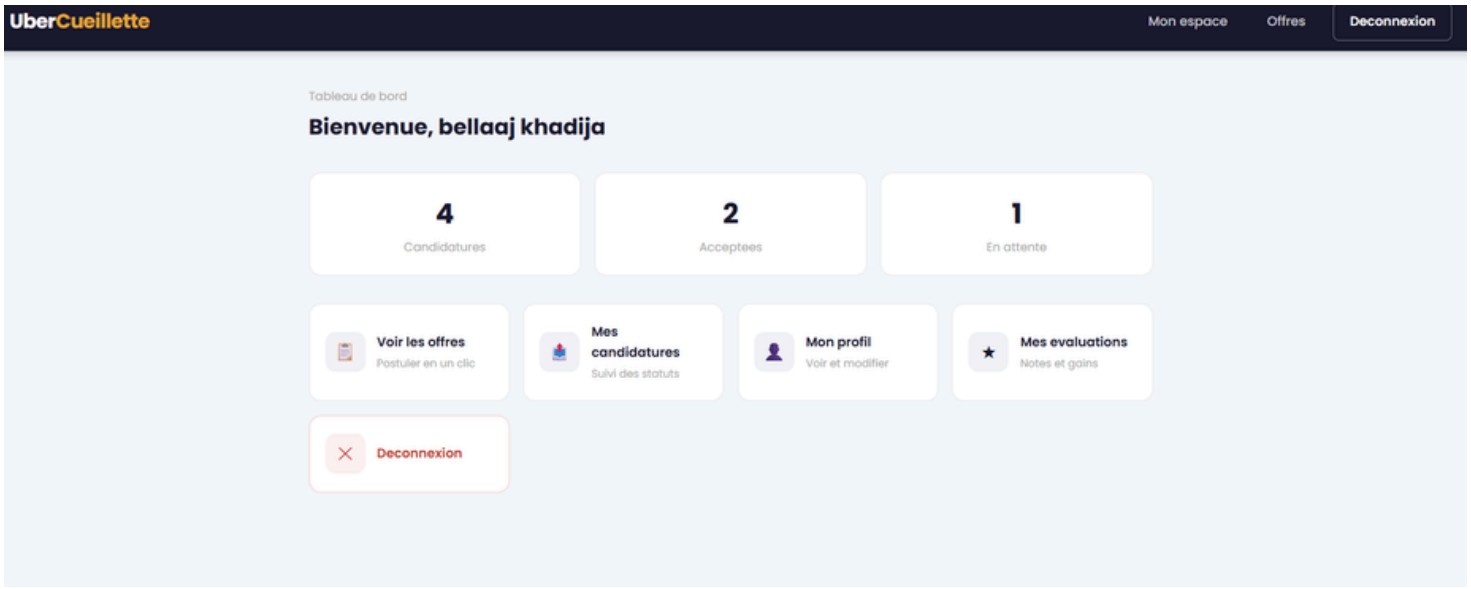
Filtre par offre. Boutons Accepter/Refuser/Evaluer selon statut. Blocage si quota atteint.



Les interfaces

Tableau de bord Ouvrier (ouvrier/menu.php):

3 compteurs de candidatures + 5 tuiles de navigation. Acces rapide aux offres, candidatures, profil et evaluations/gains.



Validation des données

Regles de validation JavaScript (script.js):

Champ / Action	Regle de validation	Note
CIN	/^[0-9]{8}$/ - exactement 8 chiffres	PHP + JS
Pseudo	/^[A-Za-z]+$/ - lettres uniquement, pas de chiffres	PHP + JS
Mot de passe	/^[A-Za-z0-9]{7,}[$#]$/ - min 8 chars, finit par $ ou #	PHP + JS
Date debut/fin	date_fin > date_debut obligatoire	PHP + JS
Date limite	date_limite strictement < date_debut	PHP + JS
Postulation	confirm() avant soumission	JS seulement
Suppression	confirm() avant suppression d'offre	JS seulement
Deconnexion	confirm() avant deconnexion	JS seulement

Analyse critique et points d'amélioration

Points forts:

Aspect	Evaluation
Architecture	Organisation claire par role, separation config/vues/logique, modules independants
Regles metier	Contraintes temporelles implementees : acceptation avant date limite, evaluation apres date fin
Base de donnees	Schéma normalise (3NF), FK avec CASCADE, 6 index de performance bien places
Securite de base	PDO prepare + bcrypt + htmlspecialchars : bonnes pratiques respectees
Design UI/UX	Interface coherente et soignee, design system CSS 14 KB, responsive mobile
Cycle fonctionnel	Cycle metier complet : inscription -> offre -> candidature -> evaluation
Extensibilite	Ajout dynamique de fruits/gouvernorats depuis l'interface utilisateur
Protection doublons	Double protection PHP + UNIQUE BDD pour les candidatures
Donnees tunisiennes	24 gouvernorats pre-charges, contexte agricole authentique et realiste

Analyse critique et points d'amélioration

Points d'amelioration:

Probleme	Solution recommandee
Incoherence ENUM SQL/PHP	Harmoniser ENUM('accepte','refuse','en_attente') ou adapter le PHP
Absence tokens CSRF	Generer un token dans la session, l'inclure dans les forms POST, verifier
Pas de reset MDP	Implementer flux email (PHPMailer + token temporaire en BDD)
Credentials BDD en clair	Utiliser getenv() avec fichier .env non versionne
Validation upload insuffisante	Remplacer par finfo_file(), limiter la taille (max 2 MB)
Pas de pagination	LIMIT/OFFSET sur lister.php et voircandidatures.php
Pas de notifications	Email ou alerte session quand candidature est traitee
Regles metier implementees	Acceptation/refus bloque apres date limite. Evaluation bloquee avant fin du chantier
Pas de recherche full-text	LIKE '%keyword%' ou FULLTEXT INDEX sur offre.adresse
Pas de logs d'activite	Table log_action pour tracer les decisions importantes
Tests unitaires absents	PHPUnit pour les fonctions de validation et queries SQL

Références

Langages et technologies

- **PHP 8.2 :**https://www.php.net
- **MySQL:**https://www.mysql.com/
- **HTML5:**https://developer.mozilla.org/fr/docs/Web/HTML
- **CSS3:**https://developer.mozilla.org/fr/docs/Web/CSS
- **JavaScript:** https://developer.mozilla.org/fr/docs/Web/JavaScript

Base de données

- **PDO (PHP Data Objects):**https://www.php.net/manual/fr/book.pdo.php
- **MySQL Functions:**https://dev.mysql.com/doc/refman/8.0/en/functions.html
- **Types de données MySQL (BLOB, ENUM...):**https://dev.mysql.com/doc/refman/8.0/en/data-types.html

Environnement de développement

- **XAMPP:**https://www.apachefriends.org/fr/index.html
- **Apache HTTP Server:** https://httpd.apache.org
- **phpMyAdmin:** https://www.phpmyadmin.net
- **Visual Studio Code:** https://code.visualstudio.com

Conclusion

UberCueillette est un mini-projet web à la fois ambitieux et abouti, qui met en évidence une bonne maîtrise des technologies PHP/MySQL ainsi que des bonnes pratiques du développement web. L'ensemble du cycle fonctionnel — de l'inscription à l'évaluation post-récolte, en passant par la publication d'offres, les candidatures et la prise de décision — est entièrement implémenté de manière cohérente, apportant une réelle valeur ajoutée au secteur agricole tunisien.

La base de données est correctement normalisée, avec des contraintes d'intégrité référentielle et des index optimisant les performances. Les aspects essentiels de sécurité (requêtes préparées via PDO, chiffrement avec bcrypt, protection via htmlspecialchars) sont bien pris en compte. L'interface se distingue par un design soigné et homogène, reposant sur un système de composants CSS interne. Par ailleurs, l'étendue fonctionnelle — couvrant les 24 gouvernorats, le cycle complet, les évaluations, la gestion de photos de profil en BLOB et des fruits dynamiques — est particulièrement remarquable pour un projet académique de ce niveau.

Avant mise en production, quelques corrections sont nécessaires : cohérence des ENUM, ajout de tokens CSRF et sécurisation des identifiants BDD. Ces points restent mineurs.

INSTITUT SUPÉRIEUR DE GESTION DE TUNIS
Since 1969

Merci

RAPPORT PRÉSENTÉ PAR :
GHOFRANE ARAAR & EYA IBN ABDELHEDI & AMIRA MIDANI

2LNBC-BIS1

ANNÉE UNIVERSITAIRE 2025/2026
DEUXIEME SEMESTRE